

Django Backend Learning — Phase 2 Notes

Database, Models, Relationships, Migrations and Django ORM

1. Django Models

A Django model is a Python class that describes data stored in the database.

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.IntegerField()
```

2. Migrations

makemigrations creates migration instructions from model changes. **migrate** applies those instructions to the database.

```
python manage.py makemigrations
python manage.py migrate
```

When you added a required address field to an existing model, Django asked for a default because existing rows needed a value for the new non-nullable column.

3. Model Relationships

We worked with Product, Customer, Order and Profile. ForeignKey connects many records to one related record. OneToOneField connects one record to one related record.

```
product = models.ForeignKey(Product, on_delete=models.CASCADE, related_name='orders')
customer = models.ForeignKey(Customer, on_delete=models.CASCADE, related_name='orders')
```

4. related_name

related_name gives a convenient name for accessing the reverse side of a relationship.

```
customer.orders.all()
```

5. Django ORM

The ORM lets Python/Django code communicate with the database using model classes and QuerySets instead of writing SQL for every basic operation.

```
Product.objects.all()
Product.objects.get(id=1)
Product.objects.filter(price=100)
```

Mental model: Python model/query code → ORM → database.

6. Basic CRUD with ORM

Common operations include creating objects, changing an object and calling save(), and deleting an object.

```
Product.objects.create(name='Laptop', price=800)
product.price = 900
product.save()
product.delete()
```

7. Django Shell

We used the Django shell to create a customer profile and work directly with model objects. The shell is useful for learning and checking database operations.

```
python manage.py shell
```

8. Revision Checklist

- I understand what a Django model represents.
- I know makemigrations vs migrate.
- I understand ForeignKey and OneToOneField.
- I understand related_name.
- I can use all(), get(), and filter().
- I understand basic create/save/delete operations.
- I know how to use the Django shell for model data.